W7 分类作业：从记录、预测到两种错误

1. 原题要你交什么

这份作业使用 `creditcard-reduced.csv`，不是 Bitcoin。第 1 问训练五种分类模型，画五张混淆矩阵，并报告准确率、精确率、召回率和 F-score；第 2 问用完整句子比较模型机制与实际表现；第 3 问固定一种分类器，比较加权前后的四指标和两张矩阵。原题内部分为 50/30/20，Canvas 是 10 points，两种分值分开记录。

完整英文原题与教师 23 格教程都保留在 正式 Notebook。前两格是身份与路径/字号设置，随后按教师原次序运行，最后从 Q1 开始是本次答卷。查看 标准导出 HTML 可以直接读到代码、运行输出和图。

2. 先认识一行记录和一个标签

一行是一条信用卡记录。原表有 32,764 行、31 列，`Class` 是要预测的类别，其余 30 列是输入。输入包括 `Time`、`V1` – `V28`、`Amount`；材料没有给出完整字段字典，本题按提供的数值列建模，不自行解释每个 V 列的业务含义。

正类（positive class）是指标要重点计算的那一类，本题规定为 `Class=1`；负类为 0。这不是说数字 1 比 0 大、类别就更严重，而是先固定指标的对象。混淆矩阵（confusion matrix）随后一直按 0、1 的顺序放行列。

原表有 177 条全列完全相同的重复记录。`drop_duplicates()` 在内存副本去掉它们后，保留 32,587 行：类别 0 有 32,114 条，类别 1 有 473 条。去重在划分前进行，避免同一条完整记录同时进入训练与测试；原 CSV 字节不改。

若两行输入相同但类别不同，就不是可以直接删掉的"全列重复"。本次数据没有发现这种冲突。将来遇到时要先核来源，不能任选一个标签。

3. 读懂最少的 Python 动作

变量是对象的名字：`x = 3` 把 3 交给名字 `x`。`raw = pd.read_csv(...)` 把 CSV 读成表。点号表示调用对象提供的功能，括号里是输入；例如 `raw.drop_duplicates()` 返回去重结果，`.copy()` 再建立独立副本。

`data['Class']` 取一列；`data.drop(columns='Class')` 得到除目标以外的输入表。把目标也放进输入会让模型直接看答案，所以 X 与 y 必须分开。这里 X 是 32,587×30 的表，y 是 32,587 个类别。

字典（dictionary）把名字映射到对象，例如 `{'tree': model}`。`for name, model in models.items():` 依次取出每个名字及模型，执行缩进的拟合和预测；五个字典条目就做五

次。`score_rows.append(...)` 保存每轮的结果，而不是覆盖前一轮。

`.fit(X_train, y_train)` 从已知输入和标签估计模型；`.predict(X_test)` 只用测试输入给出预测。`random_state=42` 固定随机过程以便复做，不表示数字 42 能提高准确率。

4. 固定训练、测试和缩放边界

训练集（training set）有 19,552 条，包含 19,268 个 0 和 284 个 1；测试集（test set）有 13,035 条，包含 12,846 个 0 和 189 个 1。`test_size=0.4` 留 40% 作测试，`stratify=y` 使两个集合都按原类别比例抽取。

分层抽样不等于把少数类复制到和多数类一样多。测试集仍然不平衡；五个模型和两棵加权比较树使用同一批测试行，误差才有可比对象。索引是原表每行的标签，`loc[train_index]` 按这些标签取行。

逻辑回归需要把不同量级的输入放到可比较的尺度。标准化（standardization）让每个值先减去训练均值，再除以训练标准差。补充小例：训练均值 10、标准差 5，新值 20 变成 $(20-10)/5=2$。测试输入继续使用训练得到的这两个量。

管道（pipeline）把缩放和模型接成一条程序。代码 `make_pipeline(StandardScaler(), LogisticRegression(...))` 的 `.fit` 只接收训练集；它先拟合缩放器，再拟合回归。不能先对整表拟合缩放再切分。树按阈值比较大小，本题的四种树模型直接使用原数值。

5. 五种模型各自在做什么

5.1 逻辑回归：先加权求和，再变成类别

逻辑回归（logistic regression）把缩放后的特征乘各自系数并相加，再加截距，得到一个分数。分数通过 sigmoid 函数变成正类概率；分数越大，概率越接近 1。它的分界面是线性的。

补充例：一个标准化输入为 2，系数为 1、截距为 −1，分数是 $2\times 1-1=1$。sigmoid 把分数 z 变成概率：

$$p=\frac{1}{1+\exp(-z)}.$$

这里指数函数 $\exp(a)=e^a$，常数 e 约为 2.718282，可以用计算器的指数键计算。分数 1 给出 $\exp(-1)\approx 0.367879$，所以 $p\approx 0.731059$。本题二分类比较两个类别概率，正类概率大于 0.5 时判为 1，因此这个例子预测 1。

若把截距改成 −3，分数变为 −1，正类概率约 0.268941，小于 0.5，预测就变为 0。本次系数由训练优化器估计，`max_iter=2000` 是最大迭代次数。其余默认设置由运行版本记录约束，未根据测试误差调参。

5.2 决策树：沿问题分支走到叶子

决策树（decision tree）在每个节点问一个特征是否小于某阈值。训练时尝试候选阈值，选择让子节点类别更集中的划分；新记录按同样条件向下走，到叶子后用其中的类别比例预测。

这里的类别混杂程度用基尼不纯度（Gini impurity）表示。若节点内两类比例是 p_0, p_1，则不纯度为 $1 - p_0^2 - p_1^2$。两类各半得到 $1 - 0.25 - 0.25 = 0.5$；全属一类得到 0。比较划分时，按子节点样本量加权这些不纯度。

`max_depth=5` 限制从根往下最多五层分裂，`min_samples_leaf=5` 要求叶子至少有五条训练记录。它们限制树把少量个例切得过细；不是规定恰好五个叶子。

5.3 随机森林与 Bagging：把多棵树的概率平均

自助采样（bootstrap sampling）从训练集有放回地抽取记录，一条记录可被抽到多次。装袋（bagging）用这些不同样本训练多棵树；本次有 100 棵，每棵深度 5、叶子至少 5 条。由于树的输入样本不同，它们的分裂也可能不同。

随机森林（random forest）进一步在每次节点分裂时随机挑部分特征作候选。本次 200 棵树，叶子至少 2 条；默认 `max_features='sqrt'`，30 个输入每次约考虑 5 个。Bagging 的基树在本题每次可考虑全部特征。

两者在这里平均树给出的类别概率，再选平均概率较大的类别。补充例：三棵树给类别 1 的概率为 0.9、0.7、0.2，均值是 0.6，于是预测 1。不是简单把任意三棵树的文字结论拼起来。

5.4 AdaBoost：让前一轮分错的记录更有影响

自适应提升（adaptive boosting, AdaBoost）逐轮拟合小树。每轮计算当前模型的加权错误率，提高分错记录在下一轮训练中的权重，并按本轮表现给模型一个投票权重。本次使用二分类 SAMME、100 轮、深度 2 的树和学习率 0.5。

补充例：加权错误率为 0.2 时，二分类模型权重为 $0.5 \ln((1 - 0.2)/0.2) = 0.693147$。这一轮分错记录的训练权重乘 $\exp(0.693147) \approx 2$。最后按各轮模型的加权投票分类。

$\ln$ 是自然对数，与 $\exp$ 互为反运算：$\ln 4$ 问的是 e 的多少次方等于 4，$\exp(\ln 4) = 4$。本例乘数也可读成 $\exp(0.5 \ln 4) = \sqrt{4} = 2$，所以分错记录的影响翻倍。

归一化就是把每条更新后的权重除以总和，让总权重回到 1。同一例子若有 5 条初始权重均为 1 的记录，1 条分错时错误率正好为 0.2。将它的权重改成 2，其他 4 条仍各为 1，总和为 6；归一化后分错记录占 $2/6$，其余每条占 $1/6$。

这与 Q3 不同：Q3 从类别比例得到一套固定权重；AdaBoost 按每条记录是否分错逐轮改变权重。教师原例的旧算法警告与本次显式 `algorithm='SAMME'` 分开记录。

6. 从矩阵亲手算出四项指标

每张图的行是真实类别、列是预测类别；左上是真负例 TN，右上是假正例 FP，左下是假负例 FN，右下是真正例 TP。FP 表示把真实 0 报成 1，FN 表示把真实 1 漏成 0。

AdaBoost 的测试矩阵为：TN=12,844、FP=2、FN=1、TP=188。先核总数 $12844 + 2 + 1 + 188 = 13035$，等于测试行数。

准确率（accuracy）是所有预测中答对的比例：

$$\frac{TP + TN}{N} = \frac{188 + 12844}{13035} \approx 0.999770.$$

精确率（precision）问"预测成 1 的记录有多少真是 1"；分母是预测 1 的 190 条：

$$\frac{TP}{TP + FP} = \frac{188}{190} \approx 0.989474.$$

召回率（recall）问"真实 1 的记录找回了多少"；分母是真实 1 的 189 条：

$$\frac{TP}{TP + FN} = \frac{188}{189} \approx 0.994709.$$

F1-score 综合精确率与召回率，用下面的等价形式代入计数更方便：

$$F_1 = \frac{2TP}{2TP + FP + FN} = \frac{376}{379} \approx 0.992084.$$

这四项均是无单位比例。代码用未舍入值计算，展示六位小数。`zero_division=0` 规定分母为零时返回 0；那是显示约定，不是产生了真实的零概率。

6.1 为什么不能只看准确率

如果全部预测为 0，12,846 条负类都答对，准确率仍有 $12846/13035 \approx 0.985501$。但 189 条正类全漏掉，召回率与 F1 都为 0。因此本题还必须看正类指标和两种错误数量。

五个模型中 AdaBoost 的 F1 最高，为 0.992084；它找回 188 个正类，误报 2 个。决策树、Bagging、AdaBoost 的最高召回率并列；正式结果同时列出这三者，不把其中一个模型写成独占最高。

7. 类别加权到底改变哪一步

类别权重（class weight）改变训练时每类记录的影响。本次训练集 0 类有 19,268 条，1 类 284 条。固定 $w_0 = 1$，令 $w_1 = 19268/284 \approx 67.845070$；程序使用未舍入的比例，使两类的总训练权重相同。

补充例：一个节点有 9 条 0 和 1 条 1，原比例为 0.9/0.1，基尼为 0.18。若给这条 1 九倍权重，模型在算不纯度时看到两类总权重各 9，比例变成各半，基尼变为 0.5。类别较少的记录

因此更能影响划分选择。

权重不复制记录、不改真实标签，也不直接把"预测阈值"改成另一个数。Q3 固定深度 5、最小叶子 5、种子 42、同一训练/测试数据，仅 `class_weight` 有无不同，才能把差异归到这一项。

7.1 如实解释没有改善的结果

两棵树都找回 188 条、漏掉 1 条，召回率同为 0.994709。加权后 FP 从 5 增加到 7，精确率从 0.974093 降到 0.964103，F1 从 0.984293 降到 0.979167。这里保留未加权树更合理，不能为迎合"加权改善"而更换测试行或隐藏结果。

8. 两个补充检查怎样帮助判断

8.1 错误成本：本次加权多出的只是误报

把一个 FP 的成本定为 1 个假设单位、一个 FN 定为 r，成本是 $FP + rFN$。r 是假设的相对成本，不是实际费用。r=10 时，未加权成本 $5 + 10 \times 1 = 15$，加权为 $7 + 10 \times 1 = 17$。

因为两模型 FN 相同、加权多 2 个 FP，本次列出的每个成本比例下都多 2 个单位；无需硬算一个不存在的正负取舍临界点。它说明本次观察到的后果，不替代对实际错误成本的调查。

8.2 Time：解释异常高的分数

训练集中 0 类的 `Time` 为 155,839–172,792，1 类为 406–170,348，分布明显不同。Q1 决策树约 0.954175 的不纯度下降来自 `Time`，说明树主要利用了这一列。这里的"重要性"不是因果效应。

进一步只把外层训练集分为 70% 拟合、30% 内部验证，用同样深度的树只看 `Time`，内部 F1 已有 0.957055。这一补充没有使用外层测试标签来选择新参数，也没有重新选择五模型的测试行。

因此高分部分来自这份给定样本的类别与 Time 分离。当前随机留出比较回答的是"同一文件随机抽取记录能否分对"；按时间预测未来记录是另一个评价设计。不要把数字高直接写成模型已在现实流程表现优异。

9. 从空内核复做

`work/` 中放 Notebook，`work/data/` 中放 `creditcard-reduced.csv` 和 `Credit_approval.csv`。后者用于保留的教师例子；本次正式三问使用前者。选择本机 `python3` 内核，Restart & Run All；第一格设置程序工作目录，不依赖别的 Notebook 变量。

```
& 'D:/anaconda3/python.exe' -X utf8 -m jupyter nbconvert --to notebook --execute --inplace --ExecutePreprocessor.kernel_name=python3 IS6400_Week7_LI_Binxiong.ipynb
& 'D:/anaconda3/python.exe' -X utf8 -m jupyter nbconvert --to html --output-dir ../submission IS6400_Week7_LI_Binxiong.ipynb
```

本次 Python 3.12.3、pandas 2.3.3、scikit-learn 1.5.1。找不到 CSV 先查工作目录与两文件名，不下载来历未知同名数据。若版本不支持 `estimator=` 或 `algorithm='SAMME'`，先核运行环境与当前签名，不靠改指标标签绕过报错。

生成的 `week7-results.json` 和两份 SVG 与 Notebook 同源。修改结果说明应改源 Notebook 并重新生成相应输出，再标准导出 HTML，不能只手改 HTML。三问更详细的课堂基础可读 [[IS6400_Business_Data_Analytics/notes/T07-分类实战-信用审批与欺诈识别|T07]]，本篇讲的是当前作业的具体数据和运行链。

10. 自测与提交

▶ 照抄教师合成例子的 class_weight={0:10,1:1}，是在照顾本题少数类吗?

▶ 若保持 TP=188、FN=1，却让 FP 从 2 增至 42，会发生什么?

手动提交时使用 W7 的 337924 入口，不要再选 Bitcoin 文件。当前 Canvas 中旧附件及原截止属于行政历史；本次只准备正确内容，未上传或替换。本人核阅、是否接受替换、实际提交和成绩另记。所有查来源、哈希、审查与文件核验信息留在 notes，不附在老师阅读的正文里。